

Setting Up Your Environment

Claude Code + Bedrock AgentCore Hackathon · Energy Symposium

Read this first — how to use this document

This document sets up your environment for the workshop. You don't have to work through it by hand.

1. **Download this document** to your machine.
2. **Upload it to Claude** in your Claude Code session (drag it into the chat, or use `/add-dir` and point at it).
3. **Ask Claude to install the items in this file** — for example: *“Read this document and install everything in it that isn't already installed on this machine. Verify each install and tell me what you installed versus what was already there.”*

That's it. This is what gets you started with **Bedrock AgentCore** — the AgentCore CLI is the one thing the lab image doesn't ship, and it's needed before you can deploy anything.

Everything below is the detail Claude needs, and a reference for you if an install fails. For prompt engineering, see the separate **Prompting Guide**. For terminology, see the **Glossary**. Both are on the hackathon portal.

What's Already in the Lab Image

Check before you install anything — most of what you need is already there:

```
claude --version           # Claude Code
aws sts get-caller-identity # AWS credentials + which account you're in
node --version && npm --version # Node 22 / npm 12
which uvx                  # uv / uvx – the Python package runner MCP servers
                           use
```

A dry run on the lab image found Claude Code, the AWS CLI (credentials already assumed via an instance role), Node 22, npm 12, and `uv` 0.12 all pre-installed. If `which uvx` returns nothing, install it once:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

What You Do Have to Install

The AgentCore CLI isn't pre-installed. It's an npm package. Install it yourself before Lab 2:

```
npm install -g @aws/agentcore
agentcore --version
```

If that fails with a permission error, don't use `sudo` — point npm at a folder you own instead:

```
npm config set prefix ~/.npm-global
export PATH="$HOME/.npm-global/bin:$PATH"
npm install -g @aws/agentcore
```

(Track 1 doesn't need this — it's Track 2 / Lab 2 only.)

Port 8080 Is Your Editor

In the lab environment, port 8080 is your browser VS Code session. If your agent's local dev server or anything else you run also binds to 8080, you'll disconnect your own editor. Use a different port for anything you run locally — for example:

```
agentcore dev --port 3000
```

MCP Servers Worth Knowing

None of these are required — the hackathon is judged on the working agent, not on which tools you used to build it. They're here for when you get stuck: each gives Claude Code live, current context instead of it guessing at an API shape, a parameter name, or a model ID.

Full catalog of 60+ AWS MCP servers: awslabs.github.io/mcp

These are Python packages, not npm packages

A common wrong turn: searching npm for `@awslabs/aws-documentation-mcp-server` and getting a 404. All three servers below are published to PyPI under names that start with `awslabs.` and run through `uvx`. The pattern for adding one to Claude Code is:

```
claude mcp add <name-you-choose> -- uvx <package-name>@latest
```

This registers the server for the current project only. Add `--scope user` to make it available in every project instead.

The three servers

1. AWS Documentation MCP Server

```
claude mcp add aws-docs -- uvx awslabs.aws-documentation-mcp-server@latest
```

- **Get it / read more:** github.com/awslabs/mcp-aws-documentation-mcp-server
- **Use it for:** looking up current AWS, Bedrock, and AgentCore API references and current model IDs instead of guessing.
- **Don't use it for:** making any actual AWS calls — it only reads documentation and cannot touch your account.

2. AWS API MCP Server

```
claude mcp add aws-api -e READ_OPERATIONS_ONLY=true -e AWS_REGION=us-east-1 \
-- uvx awslabs.aws-api-mcp-server@latest
```

- **Get it / read more:** github.com/awslabs/mcp-aws-api-mcp-server
- **Use it for:** checking what's actually deployed in your lab account (list/describe operations) before debugging blind.
- **Don't use it for:** leaving `READ_OPERATIONS_ONLY` unset in a shared lab account — without it, this server can create, modify, or delete real AWS resources.

3. Amazon Bedrock AgentCore MCP Server

```
claude mcp add agentcore-mcp -- uvx awslabs.amazon-bedrock-agentcore-mcp-server@latest
```

- **Get it / read more:** github.com/awslabs/mcp-amazon-bedrock-agentcore-mcp-server
- **Use it for:** managing Runtime, Memory, Identity, Gateway, and Policy resources through direct API tools — the fastest path through Lab 2 if you're on Track 2.
- **Don't use it for:** running your agent locally — that's still the `agentcore` CLI's job (`agentcore dev`). This MCP server manages deployed resources, it doesn't replace local development.

This one is large. It registers tool groups for Runtime, Memory, Identity, Gateway, Policy, Browser, and Code Interpreter — the browser and code-interpreter groups alone are 35 tools. Every connected tool's definition sits in the model's context for the whole session. If you're on Track 2 and already fighting for context in a multi-agent pipeline, connect it while you're working through Lab 2 and disconnect it afterwards, rather than leaving all three servers on all day.

Adding all three at once

If you'd rather configure them in one file than run three commands, create a `.mcp.json` in your project root:

```
{
  "mcpServers": {
    "aws-docs": {
      "command": "uvx",
      "args": ["awslabs.aws-documentation-mcp-server@latest"],
      "env": { "FASTMCP_LOG_LEVEL": "ERROR" }
    },
    "aws-api": {
      "command": "uvx",
      "args": ["awslabs.aws-api-mcp-server@latest"],
      "env": {
        "AWS_REGION": "us-east-1",
        "READ_OPERATIONS_ONLY": "true",
        "FASTMCP_LOG_LEVEL": "ERROR"
      }
    },
    "bedrock-agentcore": {
      "command": "uvx",
      "args": ["awslabs.amazon-bedrock-agentcore-mcp-server@latest"],
      "env": { "FASTMCP_LOG_LEVEL": "ERROR" }
    }
  }
}
```

It has to be `.mcp.json` in the project root. An `mcpServers` block in `.claude/settings.json` is silently ignored — settings files can only enable or disable servers, not define them. This is an easy hour to lose, because nothing errors: Claude Code just starts with no servers connected. `.mcp.json` is also the file to commit if you want your whole team on the same set.

Confirm they actually connected

New servers don't load into a session that's already running — exit Claude Code and start it again first. Then:

```
claude mcp list
```

Inside a session, `/mcp` shows the same list and lets you reconnect or authenticate a server without leaving the session. **Do this before you rely on a server** — pre-downloading a package proves the package exists, not that Claude Code is wired to it.

Logging in and how to submit: see the hackathon portal. **Prompt engineering and multi-agent context limits:** see the Prompting Guide.